CLOUD & DEVOPS

# Chaos Engineering Visual Guide

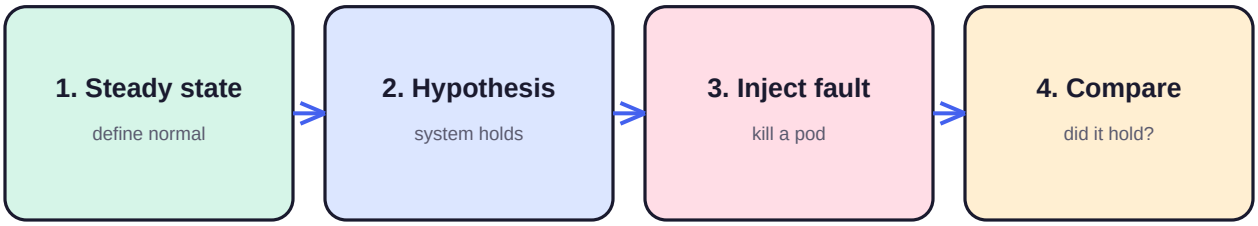
Break things on purpose —
build confidence with controlled experiments.

**ANIMATED WALKTHROUGH**
Step through pages to play the animation

STEP 1/6

# Hypothesis-Driven Experiments

Chaos is the scientific method applied to production.




Not random destruction: a hypothesis ('killing one AZ keeps p99 < 500ms') that you verify.

If you can't state the expected outcome, it's not an experiment — it's just breaking things.

STEP 2/6

# Define Steady State First

You need 'normal' measured before you can detect 'broken'.

**Steady state: checkout success 99.5% ± 0.2%, p99 < 800ms**

the hypothesis is ABOUT this number

Steady state = business metric, not CPU. Users don't care about your CPU.

- Pick 1–3 metrics that prove the system works for users.
- Measure the baseline during the experiment window (same traffic, same time).
- The experiment PASSES if steady state holds within tolerance during the fault.

STEP 3/6

# Blast Radius Control

Start tiny. Grow only with evidence.

- Abort conditions defined UP FRONT: error rate > X% → halt automatically.
- Run in production (that's where the surprises are) but during business hours first.
- Game the blast radius down, not up: one pod, one AZ, then stop and learn.

Automatic halt beats a human watching a dashboard — wire abort conditions into the experiment runner.

STEP 4/6

# Common Experiments

The greatest-hits fault injection menu.

**Kill pods**

does the deployment self-heal? (replica count holds)

**Network latency**

+200ms between services — do timeouts cascade?

**AZ failure**

lose a whole zone — does traffic shift cleanly?

**Dependency down**

payments API 500s — does checkout degrade gracefully?

Start with the boring ones: pod kills and restarts. They find the most misconfigurations per minute.

STEP 5/6

# Game Days

Scheduled, team-wide failure rehearsals.

The point isn't the failure — it's discovering that the runbook is wrong BEFORE a real outage.

- Invite everyone: devs, SRE, support, comms — incidents are team sports.
- Timebox it: 2–4 hours, business hours, with a clear abort owner.
- Every surprise becomes a ticket. No action items = wasted game day.

STEP 6/6

# Chaos Maturity Ladder

From manual pod-kills to continuous verification.

**L1: manual experiments**

engineer runs a pod-kill, watches

**L2: scheduled game days**

team rehearses quarterly scenarios

**L3: automated in CI/CD**

every deploy proves resilience

**L4: continuous in prod**

always-on fault injection, tiny blast radius

Climb one rung at a time. L4 without L1 discipline is just expensive outages.

STEP ✓/✓

# Key Takeaways

## Chaos Engineering — Remember

- Every experiment starts with a hypothesis about steady state — business metrics, not CPU.
- Blast radius grows with evidence: one pod → one AZ → (rarely) one region.
- Abort conditions are automatic and defined before the fault is injected.
- Game days rehearse people and runbooks, not just systems — every surprise becomes a ticket.
- Climb the maturity ladder: manual → scheduled → CI/CD → continuous.

**Keep exploring**

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev